

Authentication Service Security Audit

Target: Authentication Service (Signup + Login)

Stack: React + TypeScript + Supabase Auth + PostgreSQL + RLS

Reviewer: Security Audit

Status: ❌ Not Approved for Production

Executive Summary

This implementation contains several architectural and security concerns that should be addressed before production deployment.

The most significant findings are:

1. Race conditions during signup allowing duplicate account creation attempts.
2. Frontend code attempting to perform backend account orchestration.
3. Potentially broken interactions with Row Level Security (RLS).
4. Account enumeration vulnerabilities.
5. User/profile synchronization failures resulting in orphaned accounts.
6. Unhandled validation exceptions.
7. Missing brute-force protections.

Overall Production Readiness: 4/10

CRITICAL ISSUES

CRITICAL #1: Race Condition in Signup Flow

Risk Level

CRITICAL

What Is Wrong

The code performs uniqueness checks before creating records:

```
const { data: emailExists } = await supabase
  .from("users")
  .select("id")
```

```
.eq("email", validatedEmail)
.limit(1);
```

Then later:

```
await supabase.auth.signUp(...)
```

And finally:

```
await supabase.from("users").insert(...)
```

Why It's Wrong

This creates a classic Time-of-Check vs Time-of-Use (TOCTOU) race condition.

Example:

```
Request A -> username available
Request B -> username available

Request A -> insert username
Request B -> insert same username
```

Both requests pass validation before either insert occurs.

Impact

- Duplicate usernames
- Duplicate emails (depending on DB constraints)
- Signup failures under load
- Data consistency issues

Recommended Fix

Do not rely on application-level uniqueness checks.

Enforce uniqueness in PostgreSQL:

```
ALTER TABLE users
ADD CONSTRAINT users_email_unique
UNIQUE(email);

ALTER TABLE users
```

```
ADD CONSTRAINT users_username_unique  
UNIQUE(username);
```

Handle constraint violations during insert.

Verification

Run 100 concurrent signup requests using the same email and username.

Expected result:

```
1 success  
99 unique constraint violations
```

CRITICAL #2: Admin API Usage in Frontend

Risk Level

CRITICAL

What Is Wrong

```
await supabase.admin.deleteUser(authData.user.id)
```

Why It's Wrong

Admin APIs require a service role key.

Service role keys must never exist in browser code.

Possible outcomes:

1. Cleanup silently fails.
2. Service role key accidentally exposed.
3. Complete compromise of authentication system.

Impact

- Potential privilege escalation
- Orphaned accounts
- Security boundary violations

Recommended Fix

Move account creation orchestration to a backend service.

Architecture should be:

```
Frontend
  ↓
Backend API
  ↓
Supabase Admin API
```

Only backend systems should call:

```
supabase.auth.admin.deleteUser(...)
```

Verification

Search the production bundle for:

```
service_role
```

Expected result:

```
0 matches
```

CRITICAL #3: Orphaned Auth Users

Risk Level

CRITICAL

What Is Wrong

Current flow:

```
Create Auth User
  ↓
Insert Profile
  ↓
```

Insert Fails

↓

Attempt Cleanup

Cleanup is not guaranteed.

Why It's Wrong

Possible failures:

- Network interruption
- Browser refresh
- Tab close
- Supabase outage
- Permission failure

Result:

```
auth.users contains record  
public.users missing record
```

Impact

Users may authenticate successfully but permanently fail application login.

Recommended Fix

Use a PostgreSQL trigger:

```
CREATE FUNCTION create_profile_for_new_user()  
RETURNS TRIGGER AS $$  
BEGIN  
    INSERT INTO public.users (id, email)  
    VALUES (NEW.id, NEW.email);  
  
    RETURN NEW;  
END;  
$$ LANGUAGE plpgsql;
```

Attach to:

```
auth.users
```

This keeps profile creation inside the database.

Verification

Force profile insert failures and verify no orphaned users exist.

HIGH ISSUES

HIGH #1: Account Enumeration

Risk Level

HIGH

What Is Wrong

Current responses reveal account existence:

```
return { success: false, error: "Email already registered" };
```

```
return { success: false, error: "Username already taken" };
```

Why It's Wrong

Attackers can enumerate users:

```
user1@example.com  
user2@example.com  
user3@example.com
```

Responses reveal which accounts exist.

Impact

- Privacy leakage
- User enumeration
- Targeted attacks

Recommended Fix

Return generic messages:

```
return {  
  success: false,  
  error: "Unable to create account."  
};
```

Log detailed causes server-side only.

Verification

Attempt signup with known and unknown emails.

Responses should be identical.

HIGH #2: Unhandled Validation Exceptions

Risk Level

HIGH

What Is Wrong

```
SignupSchema.pick(...)  
  .parse(...)
```

Why It's Wrong

`parse()` throws exceptions.

Invalid input causes:

```
ZodError
```

No try/catch exists.

Impact

- Unhandled promise rejections
- Application crashes

- Broken user experience

Recommended Fix

Use:

```
safeParse()
```

Example:

```
const result = SignupSchema.safeParse(data);

if (!result.success) {
  return {
    success: false,
    error: "Invalid input."
  };
}
```

Verification

Submit invalid email/password combinations.

Expected:

```
Controlled validation error
```

No runtime exception.

HIGH #3: RLS Assumptions May Be Invalid

Risk Level

HIGH

What Is Wrong

Anonymous users query:


```
.from("users")  
.select("id")  
.eq("email", validatedEmail)
```

before authentication.

Why It's Wrong

Current RLS statement:

Users can only read/write their own records.

Anonymous users have no identity.

Therefore:

```
auth.uid() = null
```

The query should be denied.

Impact

Signup may fail in production.

Recommended Fix

Remove uniqueness checks from frontend.

Use:

- Database constraints
- Backend validation

Verification

Test signup while fully logged out.

Confirm RLS behavior.

HIGH #4: Auth/Profile Synchronization Failure

Risk Level

HIGH

What Is Wrong

The system maintains:

```
Supabase Auth User
+
Public Profile User
```

as separate records.

Why It's Wrong

The application assumes they remain synchronized.

Failures break that assumption.

Impact

- Authenticated users unable to access application
- Support burden
- Data integrity issues

Recommended Fix

Use:

- Database triggers
- Backend orchestration
- Automated consistency checks

MEDIUM ISSUES

MEDIUM #1: Sensitive Error Logging

Risk Level

MEDIUM

What Is Wrong

```
console.error("Auth signup error:", authError);
```

Why It's Wrong

Error objects may contain:

- Emails
- Metadata
- Internal auth information

Recommended Fix

Log minimal information in browser:

```
console.error("Authentication failed");
```

Send detailed logs only to backend monitoring.

Verification

Inspect browser console during failures.

MEDIUM #2: Username Normalization Missing

Risk Level

MEDIUM

What Is Wrong

Username are stored as entered.

Examples:

```
Alice  
alice  
ALICE
```

Impact

- User confusion
- Potential impersonation
- Duplicate identity perception

Recommended Fix

Normalize usernames:

```
const normalizedUsername =  
  validatedUsername.trim().toLowerCase();
```

Add uniqueness constraints on normalized values.

Verification

Attempt creation of:

```
Alice  
alice
```

Second registration should fail.

MEDIUM #3: Assumption of Perfect Data Integrity

Risk Level

MEDIUM

What Is Wrong

```
.single()
```

is used repeatedly.

Why It's Wrong

Database corruption or migration mistakes may create duplicate rows.

Impact

Unexpected runtime failures.

Recommended Fix

Enforce uniqueness constraints and monitor data integrity.

MEDIUM #4: Excessive Database Queries During Refresh

Risk Level

MEDIUM

What Is Wrong

Every session refresh performs:

```
.select("*")
```

Impact

Unnecessary database load.

Recommended Fix

Cache profile data and refresh only when required.

LOW ISSUES

LOW #1: Repeated Validation Logic

Risk Level

LOW

Validate entire schema at once:

```
SignupSchema.safeParse({
  email,
  password,
  username,
});
```

instead of multiple `pick()` calls.

LOW #2: Repeated User Mapping

Risk Level

LOW

The same profile mapping logic appears multiple times.

Extract:

```
function mapUserProfile(profile) {}
```

LOW #3: Logout Failure Is Hidden

Risk Level

LOW

Current implementation:

```
.catch(() => {
  console.error("Logout failed");
});
```

Caller cannot react to failures.

Return a success/failure status instead.

Answers To Review Questions

1. Can User Bypass Validation?

Partially.

Frontend validation can always be bypassed by direct API calls.

Validation must also exist server-side and/or at the database layer.

2. Can User Spoof Another User's ID?

Not directly from this code.

Assuming Supabase Auth and RLS are correctly configured, ID spoofing should not be possible.

This remains dependent on RLS policy correctness.

3. Is Database Access Restricted By RLS?

Unknown.

The signup flow appears incompatible with strict RLS because anonymous users attempt to query protected user records.

This requires verification.

4. Can Passwords Be Brute-Forced?

Potentially yes.

This code provides:

- No rate limiting
- No lockout mechanism
- No CAPTCHA
- No throttling

Security depends entirely on Supabase Auth protections.

Verify Supabase rate limiting is enabled.

5. Are All Edge Cases Handled?

No.

Missing handling includes:

- Validation exceptions
- Signup races
- Network interruption during signup
- Auth/profile desynchronization
- Orphaned users
- RLS failures

6. Does This Leak Sensitive Data?

Yes.

Current leaks include:

- Email existence
- Username existence
- Potential auth details in browser logs

7. Can This Cause Race Conditions?

Yes.

The signup uniqueness logic is race-condition vulnerable.

8. Are Errors Handled Gracefully?

Partially.

Database and auth errors are often handled.

Validation exceptions and synchronization failures are not.

Required Changes Before Production

1. Add database UNIQUE constraints for email and username.
 2. Remove client-side uniqueness checks.
 3. Remove all frontend use of `auth.admin.*`.
 4. Move account provisioning to backend or database triggers.
 5. Replace all `parse()` calls with `safeParse()`.
 6. Verify RLS compatibility with signup flow.
 7. Implement anti-enumeration responses.
 8. Verify Supabase rate limiting and brute-force protections.
 9. Add automated consistency checks between auth users and profiles.
-

Final Verdict

Deployment Recommendation: **BLOCK**

The primary architectural concern is that frontend code is attempting to coordinate creation and cleanup across both Supabase Auth and the application profile table.

That responsibility belongs in the backend or database layer.

Until the critical issues above are addressed, this implementation should not be considered production-ready.